A trace semantics for the process calculus of ProVerif

Artie Khovanov with lots of assistance from Claude Fable

ProVerif is a tool for reasoning about cryptographic security in a symbolic model with a Dolev-Yao adversary. Although it uses resolution of Horn clauses to find attacks, it is backed theoretically by a process calculus, which is a variant of the applied pi calculus. We use the specification of ProVerif's semantics given in Blanchet 16 [1].

We formalise the ProVerif process calculus in Lean. Rather than copying the specification, we use a shallow translation in order to make security proofs simpler down the line. This means that we make use of existing Lean syntax and constructs where possible. In particular, some invariants are enforced only by what terms can arise from translating a valid ProVerif program. We also deviate from the calculus when sticking exactly to the specification would complicate the design.

We use a trace-based semantics to encode the possible outcomes of parallel processes communicating on a network. The adversary is modelled as a set of traces satisfying informational constraints. Our trace semantics are good enough to express secrecy and simple ProVerif queries relating events. However, it is strictly less expressive than the formal operational semantics of ProVerif; in particular, we cannot express observational equivalence.

We give a mathematical account of the Lean development, and of the formal translation from terms in ProVerif's process calculus to our traces. Everything has been formalised in Lean.

1 The trace semantics

1.1 Messages

First, we define the set of messages. These are built from abstract constructors and subject to equations as well as destructor functions.

Fix a supply of variables $\{\nu_k \mid k \in \mathbb{N}\}$ and a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of arity-n constructor symbols f. The set $\mathcal{T}$ of *constructor terms* t is built up inductively by

$$t ::= \nu_k \mid f(t_1, \ldots, t_n),$$

where $f \in \Sigma_n$ $(n \in \mathbb{N})$.

In order to define deterministic destructors later, we must restrict the kinds of equations that may appear between constructor terms.

Fix a set of *free* constructors. A set of equations $E \subseteq \mathcal{T} \times \mathcal{T}$ is *valid* if, for every $(l, r) \in E$, both l and r are of the form $f(\ldots)$ with f not free.

A *substitution* on terms is a homomorphic extension of a valuation $\{\nu_k\} \to \mathcal{T}$ on variables. Given a set of equations E, define $\equiv_E$ to be the least equivalence relation on constructor terms that contains E and is closed under substitutions and constructor applications.

The free constructors are the ones which may be used when destructor patterns. In other words, the head symbols on either side of an equation cannot appear in any destructor's patterns.

Definition 1.1 (Messages)**.** Fix everything as before. Given a valid set of equations E, the set of *messages* is $\mathcal{M} = \mathcal{T}/\equiv_E$.

The constants ν_k and constructors f descend to $\mathcal{M}$ in the obvious way, and every substitution on constructor terms descends uniquely to one on messages.

Adjoin a symbol $\perp$ to $\mathcal{M}$ to represent the failure of a computation; for $g : \mathcal{M}^n \to \mathcal{M}$, write $g^\uparrow$ for the extension to $\mathcal{M}_\perp^n \to \mathcal{M}_\perp$ which returns $\perp$ when any argument is $\perp$.

We now come to destructors, whose definition is a bit fiddly. The details are ultimately unimportant to the overall model.

Definition 1.2 (Destructors)**.** A *pattern* p is given by

$$p ::= \nu_k \mid \perp \mid \lceil T \rceil,$$

where $T \in \mathcal{T}$. We extend a valuation $\theta : \{\nu_k\} \to \mathcal{M}_\perp$ to a pattern in the obvious way, except that we only define the extension to $\lceil T \rceil$ if $\theta(\nu_k) \neq \perp$ for all ν_k appearing in T. We say θ *matches* p with $u \in \mathcal{M}_\perp$ if $\theta(p) = u$.

A *destructor* $g : \mathcal{M}_\perp^n \to \mathcal{M}_\perp$ is given by a list of rules $g(\vec{p}_1) \to q_1;\ \ldots;\ g(\vec{p}_k) \to q_k$ such that

1. each constructor on the left-hand side is free,
2. each variable on the right-hand side of a rule occurs on its left-hand side, with those under $\lceil \cdot \rceil$ on the right-hand side also occurring under $\lceil \cdot \rceil$ on the left-hand side.

As a function, g is defined by

$$g(\vec{u}) = \begin{cases} \theta(q_i) & i \text{ least such that some } \theta \text{ matches } \vec{p}_i \text{ with } \vec{u}, \\ \perp & \text{otherwise.} \end{cases}$$

Destructors are well-defined because, by construction, there is at most one valuation matching a pattern against a (possibly failing) term.

Intuitively, the pattern ν_k represents a possibly-failing message $u \in \mathcal{M}_\perp$, $\lceil \nu_k \rceil$ represents a non-failing message $m \in \mathcal{M}$, $\perp$ represents $\perp$, and $\lceil f(t_1, \ldots, t_n) \rceil$ represents $f(m_1, \ldots, m_n) \in \mathcal{M}$ such that repeated variables are substituted the same.

Definition 1.3 (Public operations)**.** Fix a set of public constructors $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and public destructors Dt. The *public operations* $\mathcal{M}_\perp^n \to \mathcal{M}_\perp$ are $\mathrm{Ops} = \{f^\uparrow \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt}$. Write $\langle S \rangle$ for the closure of $S \subseteq \mathcal{M}$ under public operations (possibly involving failing terms).

Our adversary will grow its knowledge by applying public operations to the messages it knows. Note that this automatically includes public constant constructors.

1.2 Traces and processes

In our algebraic model of cryptographic protols, processes run computations on messages in parallel and exchange them over a hostile network. We represent a process by its possible traces: sequences of events which characterise the security properties of that run.

Definition 1.4 (Traces)**.** An *action* is given by

$$a \coloneqq \mathsf{out}(c, m) \mid \mathsf{in}(c, m) \mid \mathsf{comm}(c, m) \mid \mathsf{new}(\nu_k) \mid \mathsf{ev}(m),$$

where $c, m \in \mathcal{M}$ and $k \in \mathbb{N}$.

A *trace* is list of actions; write ε for the empty trace and and $s \leq t$ if s is a prefix of t. A trace is *closed* if it does not contain any "unfinished" actions $\mathsf{out}(c, m)$ or $\mathsf{in}(c, m)$. A *process* is a prefix-closed set of traces.

Write Act for the set of actions, Tr for the set of traces, and Proc for the set of processes.

The actions in and out represent one side of a message transmission, while comm represents a completed communication between parallel sub-processes. The action new represents the creation of a nonce, and is necessary to prevent nonce collision when running processes in parallel. The action ev allows a process to raise an arbitrary event flag – it's used to signal things like key leaks.

Definition 1.5 (Process constructors)**.** We define the following standard processes (it is easy to check the sets of traces remain prefix-closed). For an action a, a process P, and a map $F : \mathcal{M} \to \text{Proc}$:

$$\begin{aligned} \mathbf{0} &= \{\varepsilon\}, & a \cdot P &= \{\varepsilon\} \cup \{a\, t \mid t \in P\}, \\ \mathsf{out}(c, m).P &= \mathsf{out}(c, m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c, m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(\nu_k) \cdot \{t \in F(\nu_k) \mid k \notin \text{nonces}(t)\}. \end{aligned}$$

A trace of a process should be interpreted as a **partial** execution of that process in an arbitrary network: arbitrary messages may be received from the adversary, outgoing messages may be intercepted by the adversary, and the process may stop at any time. It remains to define parallel composition of processes, which is a bit more involved due to the need to keep track of completed communications.

Fix an index set I. Given a "tagged trace" $s \in (I \times \text{Act})^*$ and $i \in I$, write $s{\restriction}i \in \text{Tr}$ for the sub-trace of actions tagged i and $\bar{s} \in \text{Tr}$ for the trace with the tags forgotten. For a family of traces $\vec{t} : I \to \text{Tr}$, the tagged trace s *interleaves* the t_i if $s{\restriction}i = t_i$ for all i.

We say a tagged trace s *reduces to* the trace t if it satisfies $s \to t$, the smallest relation such that $s \to \bar{s}$ and, whenever $s \to t$ and $i \neq j$,

$$(i, \mathsf{out}(c, m))\,(j, \mathsf{in}(c, m))\, s \to \mathsf{comm}(c, m)\, t.$$

Definition 1.6 (Parallel composition)**.** Given a family $P : I \to \text{Proc}$ of processes, a trace t lies in their parallel composition $\text{merge}_I(P)$ if it has no repeated occurences of $\mathsf{new}(\nu_k)$ and there is a family of traces $\vec{u}$ and a tagged trace s such that $u_i \in P_i$ for all i, s interleaves $\vec{u}$, and s reduces to t.

We write $P \mid Q = \text{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \text{merge}_{\mathbb{N}}(k \mapsto P)$. We can prove that $!P = P \mid !P$.

A trace in the parallel composition of several processes represents one possible parallel execution of these processes. Keep in mind that, since messages may be intercepted by the adversary, a communication between two parallel processes may, or may not, be completed.

1.3 The adversary and the security properties

The adversary is itself modelled as a process, run in parallel with the protocol under investigation. We keep track of the adversary's knowledge to ensure attacks are valid.

Definition 1.7 (Adversary process)**.** The adversary processes $\text{Adversary}(K) \in \text{Proc}$, for $K \subseteq \mathcal{M}$, are defined jointly and inductively by

1. $\varepsilon \in \text{Adversary}(K)$;
2. if $c \in \langle K \rangle$ and $e \in \text{Adversary}(K \cup \{m\})$ then $\mathsf{in}(c, m)\, e \in \text{Adversary}(K)$;
3. if $c \in \langle K \rangle$, $m \in \langle K \rangle$ and $e \in \text{Adversary}(K)$ then $\mathsf{out}(c, m)\, e \in \text{Adversary}(K)$;
4. if $c \in \langle K \rangle$, $m \in \langle K \rangle$ and $e \in \text{Adversary}(K)$ then $\mathsf{comm}(c, m)\, e \in \text{Adversary}(K)$;
5. if $\nu_k \notin \text{nonces}(e)$ and $e \in \text{Adversary}(K \cup \{\nu_k\})$ then $\mathsf{new}(\nu_k)\, e \in \text{Adversary}(K)$.

Definition 1.8 (Attacks)**.** Let P be a process. An *attack* on P is a closed trace of the process $P \mid \text{Adversary}(\emptyset)$. Write $\text{Attacks}(P)$ for the set of attacks on P.

The adversary starts knowing nothing, and can only communicate on channels it already knows about (typically, the set of public channels is fixed). The closed trace condition finally forces all the communications in the process run to be completed – in other words, we model the adversary intercepting and supplying messages as required. Note that $\text{Attacks}(P)$ is prefix-closed.

We can now define security properties of a process. First, we need to keep track of what the adversary has heard.

Definition 1.9 (Knowledge)**.** Let $K \subseteq \mathcal{M}$, and let t be a trace. The adversary *knows* the message m after t starting from K if $m \in \langle K \rangle(t)$, defined inductively by

$$\begin{aligned} \langle K \rangle(\varepsilon) &= \langle K \rangle; \\ \langle K \rangle(\mathsf{comm}(c, m)\, t) &= \langle \langle K \cup \{m\} \rangle(t) \rangle && \text{if } c \in \langle K \rangle(t); \\ \langle K \rangle(a\, t) &= \langle K \rangle(t) && \text{otherwise.} \end{aligned}$$

Definition 1.10 (Event correspondences)**.** A *fact* is given by

$$f ::= \mathsf{event}(e) \mid \mathsf{knows}(m),$$

where $e, m \in \mathcal{M}$. We say $\mathsf{event}(e)$ *holds* for a trace t if $\mathsf{ev}(e)$ appears in t, and $\mathsf{knows}(m)$ *holds* for t if the adversary knows m after t (starting from $\emptyset$).

Fix a process P, and let f_1 and f_2 be facts. Then the *correspondence* $f_1 \implies f_2$ holds for P if, given any attack on P, for every substitution σ on messages such that $\sigma(f_1)$ holds, there is some substitution τ such that $\sigma(f_1) = \tau(f_1)$ and $\tau(f_2)$ holds.

The substitutions allow us to parameterise events. A correspondence $f_1 \implies f_2$ states that some $\sigma'(f_2)$ holds by the time $\sigma(f_1)$ does, because the set of attacks is prefix closed.

Similarly, we say that the message m is *secret* in P if $\mathsf{knows}(m)$ doesn't hold for any attack on P.

2 The calculus of ProVerif

We set out the process calculus of ProVerif as described in Blanchet's survey (§2.1, §2.4, §2.5.1, §3.1–3.3). The survey states §3 for the core calculus; we extend to include equations by taking its definitions modulo $=_E$.

2.1 Syntax

Definition 2.1 (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\text{pub}} \uplus \mathcal{N}_{\text{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail}, \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(N, M); P \mid \mathsf{in}(N, x); P \mid P \mid Q \mid !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}(M); P, \end{aligned}$$

where f ranges over constructors and h over constructors and destructors. Types are erased.

Definition 2.2 (Rewrite rules)**.** Fix a set of *may-fail variables* $u, v, \ldots$, disjoint from the variables of Definition 2.1, which are the *ordinary variables*. A *may-fail term* is

$$U ::= M \mid u \mid \mathsf{fail},$$

where M is a term without names. As in the survey's §2.1, a destructor g of arity n is defined by a finite ordered list $\mathrm{def}(g)$ of *rewrite rules* $g(U_1, \ldots, U_n) \to U$ between may-fail terms without names, the variables of U occurring in $U_1, \ldots, U_n$. As in the survey's §2.4, each constructor f has $\mathrm{def}(f)$ containing the identity rule $f(\vec{x}) \to f(\vec{x})$, and every $\mathrm{def}(h)$ ends with the rule $h(\vec{u}) \to \mathsf{fail}$ for may-fail variables $\vec{u}$, which makes h fail when no other rule applies. A *substitution* maps ordinary variables to terms and may-fail variables to may-fail terms.

An equational theory E is a finite set of equations between terms without names; $=_E$ is the closure of E under reflexivity, symmetry, transitivity, application of function symbols and substitution.

Definition 2.3 (Model)**.** A *model* consists of a finite set of function symbols, each a constructor or a destructor and each public or private, every destructor g given by rules $\mathrm{def}(g)$ as in Definition 2.2; an equational theory E; finite disjoint sets $\mathcal{N}_{\text{pub}}$, $\mathcal{N}_{\text{priv}}$ of free names; and a closed process P_0 with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\text{pub}} \uplus \mathcal{N}_{\text{priv}}$.

Definition 2.4 (Queries)**.** A *query fact* is $\mathsf{event}(M)$ or $\mathsf{attacker}(M)$ with M a term over constructors, free names and a finite set X of variables; it is an *event fact* or an *attacker fact* accordingly. A *query* is $\mathsf{query}\ \mathsf{attacker}(M)$ with M without variables, or $\mathsf{query}\ F \implies H$ with F and H query facts over X.

2.2 Semantics

Evaluation $D \Downarrow U$ of closed expressions to closed may-fail terms is the least relation with $M \Downarrow M$, $\mathsf{fail} \Downarrow \mathsf{fail}$, and $h(D_1, \ldots, D_n) \Downarrow \sigma U'_j$ whenever $D_i \Downarrow U_i$ for all i, $\mathrm{def}(h) = (h(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \le k}$, $\sigma U'_{j,i} =_E U_i$ for all i, and for all $i < j$ and all σ' there is some l with $\sigma' U'_{i,l} \neq_E U_l$.

A *configuration* is $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P}$ with $\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}$ finite disjoint sets of names and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$. Writing $\mathcal{E}$ for the pair of name sets, reduction is the least relation closed under

$$\mathcal{E}, \mathcal{P} \cup \{\mathbf{0}\} \to \mathcal{E}, \mathcal{P} \qquad \text{(Red Nil)}$$

$$\mathcal{E}, \mathcal{P} \cup \{P \mid Q\} \to \mathcal{E}, \mathcal{P} \cup \{P, Q\} \qquad \text{(Red Par)}$$

$$\mathcal{E}, \mathcal{P} \cup \{!P\} \to \mathcal{E}, \mathcal{P} \cup \{P, !P\} \qquad \text{(Red Repl)}$$

$$(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P} \cup \{\mathsf{new}\ a; P\} \to (\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}} \cup \{a'\}), \mathcal{P} \cup \{P\{a'/a\}\} \quad \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}} \qquad \text{(Red Res)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} \to \mathcal{E}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} \quad \text{if } N =_E N' \qquad \text{(Red I/O)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\{M/x\}\} \quad \text{if } D \Downarrow M \qquad \text{(Red Eval 1)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } D \Downarrow \mathsf{fail} \qquad \text{(Red Eval 2)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \quad \text{if } M =_E \mathsf{true} \qquad \text{(Red Cond 1)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } M \neq_E \mathsf{true} \qquad \text{(Red Cond 2)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{event}(M); P\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \qquad \text{(Red Event)}$$

A *trace* is a finite reduction sequence $\mathcal{E}_0, \mathcal{P}_0 \to \cdots \to \mathcal{E}_n, \mathcal{P}_n$, written $\mathcal{E}_0, \mathcal{P}_0 \to^* \mathcal{E}_n, \mathcal{P}_n$.

Definition 2.5 (Survey, Definitions 3.1–3.5)**.** Let P_0 be a closed process.

1. A closed process Q is an $\mathcal{N}_{\mathrm{pub}}$-*adversary* if $\mathrm{fn}(Q) \subseteq \mathcal{N}_{\mathrm{pub}}$, every function symbol of Q is public, and Q contains no event.
2. A trace *outputs* M *publicly* if it contains a step (Red I/O) whose output is $\mathsf{out}(N, M'); Q$ with $N \in \mathcal{N}_{\mathrm{pub}}$ and $M' =_E M$. It *executes* $\mathsf{event}(M)$ if it contains a step (Red Event) on $\mathsf{event}(M'); P$ with $M' =_E M$.
3. P_0 *preserves the secrecy of* the closed term M *from* $\mathcal{N}_{\mathrm{pub}}$ if, for some $\mathcal{N}_{\mathrm{priv}}$ disjoint from $\mathcal{N}_{\mathrm{pub}}$ with $\mathrm{fn}(M) \cup \mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, no trace $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\} \to^* \mathcal{E}', \mathcal{P}'$ with Q an $\mathcal{N}_{\mathrm{pub}}$-adversary outputs M publicly.
4. P_0 *satisfies* $\mathsf{event}(M) \rightsquigarrow \mathsf{event}(M')$ *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries* if, for some $\mathcal{N}_{\mathrm{priv}}$ as above, for every $\mathcal{N}_{\mathrm{pub}}$-adversary Q, every trace Tr from $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\}$ and every substitution σ: if Tr executes $\mathsf{event}(\sigma M)$ then there is some σ' with $\sigma' M =_E \sigma M$ such that Tr executes $\mathsf{event}(\sigma' M')$.

Fix $\mathcal{N}_{\mathrm{priv}}$ as in Definition 2.5, and for an $\mathcal{N}_{\mathrm{pub}}$-adversary Q write $\mathcal{C}_Q$ for the configuration $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\}$.

Definition 2.6 (Satisfaction)**.** A trace *satisfies* $\mathsf{attacker}(M)$ if it outputs M publicly, and *satisfies* $\mathsf{event}(M)$ if it executes $\mathsf{event}(M)$.

Definition 2.7 (Correspondences between facts)**.** Let F and H be query facts over X. P_0 *satisfies* $F \rightsquigarrow H$ *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries* if for every $\mathcal{N}_{\mathrm{pub}}$-adversary Q, every trace Tr from $\mathcal{C}_Q$

and every substitution σ: if Tr satisfies σF then there is some σ' with $\sigma' F =_E \sigma F$ such that Tr satisfies $\sigma' H$. For event facts this is Definition 2.5(4).

3 The translation

We now translate the calculus of ProVerif into the trace semantics: a destructor given by rewrite rules becomes a destructor given by patterns, an expression a possibly-failing message, a process a set of traces, and a query a security property. We then relate the translation to the operational semantics of the survey.

Throughout, fix a model (Definition 2.3). Identify the ordinary variables x and the may-fail variables u of Definition 2.2 with distinct variables ν_x and ν_u of the trace semantics, so that a term without names is a constructor term. The trace semantics is instantiated from the model: its constructors Σ are the constructors of the model together with the free names as nullary constructors, its free constructors Fr are the constructors at the root of no equation of E, and its equations are those of E. For E to be valid we require

(R1) neither side of an equation of E is a variable.

Definition 3.1 (Destructors from rewrite rules)**.** A may-fail term U without names gives the pattern $\widetilde{U}$ of Definition 1.2 with

$$\widetilde{M} = \lceil M \rceil, \qquad \widetilde{u} = \nu_u, \qquad \widetilde{\mathsf{fail}} = \bot.$$

A destructor g with $\mathrm{def}(g) = (g(U_{i,1}, \ldots, U_{i,n}) \to U_{i,0})_{i \le k}$ denotes the destructor of Definition 1.2 given by the rules

$$g(\widetilde{U}_{1,1}, \ldots, \widetilde{U}_{1,n}) \to \widetilde{U}_{1,0};\ \ldots;\ g(\widetilde{U}_{k,1}, \ldots, \widetilde{U}_{k,n}) \to \widetilde{U}_{k,0},$$

provided these satisfy the two conditions of Definition 1.2. The second always holds: every variable on the right-hand side of a rule occurs on its left-hand side by Definition 2.2, under $\lceil \cdot \rceil$ on both sides if it is ordinary and on neither side if it is may-fail. The first says that the terms among the arguments of every rule of $\mathrm{def}(g)$ have all their constructors in Fr. The final rule $g(\vec{u}) \to \mathsf{fail}$ of $\mathrm{def}(g)$ becomes $g(\nu_{u_1}, \ldots, \nu_{u_n}) \to \bot$, which matches every argument, so the destructor returns $\bot$ whenever no earlier rule matches, as Definition 1.2 has it in any case.

Every destructor of the model, public or private, is read by Definition 3.1; for this we require

(R2) in every rule of every destructor, the terms among the arguments have all their constructors in Fr.

The public constructors Pub of the trace semantics are the public constructors of the model together with the names in $\mathcal{N}_{\mathrm{pub}}$, and its public destructors Dt are the public destructors of the model. These conditions exclude some ProVerif models: an equation with a variable as one side, such as the survey's (2.8), violates (R1), and a destructor whose rules match on a symbol at the root of an equation has no counterpart under (R2).

3.1 Expressions, processes and queries

Definition 3.2 (Translation)**.** An *environment* ρ maps variables and names to $\mathcal{M}$; ρ_0 maps each free name a to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension.

Expressions are evaluated in $\mathcal{M}_\perp$:

$$[\![M]\!]\rho = \rho M, \qquad [\![\mathsf{fail}]\!]\rho = \perp, \qquad [\![f(\vec{D})]\!]\rho = f^\uparrow([\![\vec{D}]\!]\rho), \qquad [\![g(\vec{D})]\!]\rho = g([\![\vec{D}]\!]\rho)$$

for a constructor f and a destructor g, the latter the destructor of Definition 3.1. Processes are translated by

$$\begin{aligned}
[\![\mathbf{0}]\!]\rho &= \mathbf{0}, & [\![\mathsf{out}(N,M);P]\!]\rho &= \mathsf{out}(\rho N, \rho M).[\![P]\!]\rho,\\
[\![\mathsf{event}(M);P]\!]\rho &= \mathsf{ev}(\rho M).[\![P]\!]\rho, & [\![\mathsf{in}(N,x);P]\!]\rho &= \mathsf{in}(\rho N).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big),\\
[\![P \mid Q]\!]\rho &= [\![P]\!]\rho \mid [\![Q]\!]\rho, & [\![\mathsf{new}\ a;P]\!]\rho &= \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big),\\
[\![!P]\!]\rho &= !\,[\![P]\!]\rho, & [\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho &= \begin{cases} [\![P]\!]\rho[x \mapsto m] & [\![D]\!]\rho = m \in \mathcal{M},\\ [\![Q]\!]\rho & [\![D]\!]\rho = \perp, \end{cases}\\
& & [\![\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho &= \begin{cases} [\![P]\!]\rho & \rho M = \mathsf{true},\\ [\![Q]\!]\rho & \rho M \neq \mathsf{true}. \end{cases}
\end{aligned}$$

The translation of P_0 is $[\![P_0]\!]\rho_0$. A translated process mentions no ν_k except through new.

Remark 3.3. In the development a process is a program in the continuation monad with answer type Proc, run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c,m) = \lambda\kappa.\,\mathsf{out}(c,m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation.

Definition 3.4 (Translation of queries)**.** Identify the variables X of a query with distinct variables ν_x of the trace semantics, and let ρ_X extend ρ_0 by $\rho_X(x) = \nu_x$. A query fact over X translates to the fact

$$[\![\mathsf{event}(M)]\!] = \mathsf{event}(\rho_X M), \qquad [\![\mathsf{attacker}(M)]\!] = \mathsf{knows}(\rho_X M).$$

The query $\mathsf{query}\ \mathsf{attacker}(M)$ denotes that $\rho_0 M$ is secret in $[\![P_0]\!]\rho_0$, and $\mathsf{query}\ F \Longrightarrow H$ that the correspondence $[\![F]\!] \implies [\![H]\!]$ holds for $[\![P_0]\!]\rho_0$.

3.2 The correspondence

Names of the reduction semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through a bijection ι between the names outside $\mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, those that (Red Res) may create, and $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota\vec{M})$ and $\iota\,\mathsf{fail} = \perp$; $\iota M = \iota M'$ exactly when $M =_E M'$. Let ρ_ι be the environment with $\rho_\iota(a) = \iota a$ for every free or created name a.

Lemma 3.5 (Evaluation)**.** *For every closed expression D over free and created names, some U satisfies $D \Downarrow U$, and $D \Downarrow U$ implies $\iota U = [\![D]\!]\rho_\iota$.*

Definition 3.6 (Labelled semantics)**.** A *labelled configuration* is $(U, \mathcal{P})$ with $U \subseteq \mathbb{N}$ finite and $\mathcal{P}$ a finite multiset of closed processes whose names are free names and symbols ν_k. Let ρ_ν be the environment extending ρ_0 by $\rho_\nu(\nu_k) = \nu_k$, so that a closed term M over such names denotes $\rho_\nu M \in \mathcal{M}$. The τ-steps $(U, \mathcal{P}) \xrightarrow{\tau} (U, \mathcal{P}')$ are the steps (Red Nil), (Red Par), (Red Repl),

(Red Eval 1), (Red Eval 2), (Red Cond 1) and (Red Cond 2), with $=_E$ read on $\mathcal{M}$. The visible steps are

$$\begin{aligned}
(U, \mathcal{P} \cup \{\mathsf{out}(N, M); P\}) &\xrightarrow{\mathsf{out}(\rho_\nu N, \rho_\nu M)} (U, \mathcal{P} \cup \{P\}),\\
(U, \mathcal{P} \cup \{\mathsf{in}(N, x); P\}) &\xrightarrow{\mathsf{in}(\rho_\nu N, \rho_\nu M)} (U, \mathcal{P} \cup \{P\{M/x\}\})\\
&\qquad \text{for every closed term } M,\\
(U, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\}) &\xrightarrow{\mathsf{comm}(\rho_\nu N, \rho_\nu M)} (U, \mathcal{P} \cup \{Q,\ P\{M/x\}\})\\
&\qquad \text{if } N =_E N',\\
(U, \mathcal{P} \cup \{\mathsf{new}\ a; P\}) &\xrightarrow{\mathsf{new}(\nu_k)} (U \cup \{k\}, \mathcal{P} \cup \{P\{\nu_k/a\}\}) \quad \text{for every } k \notin U,\\
(U, \mathcal{P} \cup \{\mathsf{event}(M); P\}) &\xrightarrow{\mathsf{ev}(\rho_\nu M)} (U, \mathcal{P} \cup \{P\}).
\end{aligned}$$

The third step is (Red I/O) between two members of $\mathcal{P}$. It is visible rather than a τ-step because a trace records a completed communication, which is what the adversary's knowledge $\langle K \rangle(t)$ is read off. With $\stackrel{\alpha}{\Longrightarrow} = \stackrel{\tau}{\rightarrow}^* \stackrel{\alpha}{\rightarrow} \stackrel{\tau}{\rightarrow}^*$, the set of *weak traces* is

$$\mathrm{Tr}(U, \mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Tr} \mid (U, \mathcal{P}) \stackrel{\alpha_1}{\Longrightarrow} \cdots \stackrel{\alpha_n}{\Longrightarrow}\}.$$

Theorem 3.7 (Adequacy)**.** *For every closed process P whose names are free names and symbols ν_k, $[\![P]\!]\rho_\nu = \mathrm{Tr}(\emptyset, \{P\})$. For all finite multisets $\mathcal{P}, \mathcal{Q}$ of such processes,* $\mathrm{Tr}(\emptyset, \mathcal{P} \uplus \mathcal{Q}) = \mathrm{Tr}(\emptyset, \mathcal{P}) \mid \mathrm{Tr}(\emptyset, \mathcal{Q})$.

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$. For a trace Tr from $\mathcal{C}_Q$ let $\mathrm{outputs}(\mathrm{Tr}) \subseteq \mathcal{M}$ be the set of ιM for the terms M that Tr outputs publicly, and for $t \in \mathrm{Tr}$ say that Tr and t *have the same events* if Tr executes $\mathsf{event}(M)$ exactly when $\mathsf{ev}(\iota M)$ occurs in t.

Theorem 3.8 (Attacks)**.** *Fix an $\mathcal{N}_{\mathrm{pub}}$-adversary Q and a trace* Tr *from $\mathcal{C}_Q$, as well as a trace $t \in \mathcal{A}_0$.*

(E) (i) There is $t' \in \mathcal{A}_0$ with the same events as Tr*.*

(ii) There are an $\mathcal{N}_{\mathrm{pub}}$-adversary Q' and a trace Tr$'$ *from $\mathcal{C}_{Q'}$ with the same events as t.*

(K) Suppose further $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$.

(i) For every $m \in \mathrm{outputs}(\mathrm{Tr})$ there is t' as in (E)(i) such that $m \in \langle \emptyset \rangle(t')$.

(ii) For every $m \in \langle \emptyset \rangle(t)$ there are Q' and Tr$'$ *as in (E)(ii) such that $m \in \mathrm{outputs}(\mathrm{Tr}')$.*

Theorem 3.9 (Secrecy)**.** *Let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$, let M be a closed term with $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$ and let $s = \rho_0 M$. Then s is secret in $[\![P_0]\!]\rho_0$ if and only if P_0 preserves the secrecy of M from $\mathcal{N}_{\mathrm{pub}}$.*

Remark 3.10. The hypothesis $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ cannot be dropped: when $\mathcal{N}_{\mathrm{pub}} = \emptyset$ no trace outputs anything publicly, so P_0 preserves the secrecy of every M, whereas $\rho_0 M$ is not secret in $[\![P_0]\!]\rho_0$ whenever $\rho_0 M \in \langle \emptyset \rangle$, for instance for a public constant M.

Theorem 3.11 (Correspondences)**.** *Let F and H be query facts over X such that H is an event fact, and let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ if F is an attacker fact. Then P_0 satisfies $F \rightsquigarrow H$ against $\mathcal{N}_{\mathrm{pub}}$-adversaries if and only if the correspondence $[\![F]\!] \implies [\![H]\!]$ holds for $[\![P_0]\!]\rho_0$.*

Remark 3.12. Neither hypothesis can be dropped. When $\mathcal{N}_{\text{pub}} = \emptyset$ and $F = \mathsf{attacker}(M)$, P_0 satisfies the correspondence vacuously, whereas $[\![F]\!] \implies [\![H]\!]$ fails for $[\![P_0]\!]\rho_0$ as soon as $\sigma(\rho_X M) \in \langle\emptyset\rangle$ for some substitution σ. For $H = \mathsf{attacker}(M)$, Definition 2.6 requires σM itself to be output publicly, whereas $\mathsf{knows}(\sigma M)$ holds as soon as the adversary can compute σM from the messages it has heard: with the symmetric encryption of the survey's §2.1, $\mathcal{N}_{\text{pub}} = \{c\}$ and $P_0 = \mathsf{out}(c, \mathsf{senc}(s, k)); \mathsf{out}(c, k); \mathsf{event}(e)$, the correspondence $\mathsf{event}(e) \implies \mathsf{knows}(s)$ holds for $[\![P_0]\!]\rho_0$ while the correspondence of Definition 2.7 fails. The second reading is ProVerif's.

References

[1] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016. `../Reference/BlanchetFnTPS16.pdf`, doi:10.1561/3300000004.